

Guía de actividades: excepciones sobre CLÍNICA KETEKURA

Sesión 1.3.1 · Indicador de logro 1.3 · 4 ejercicios · Documento del estudiante

Asignatura	BDY1103 – Taller de Base de Datos	Experiencia	EA1
Tiempo	50 min en clase + trabajo autónomo	Modalidad	Individual
Herramienta	Oracle SQL Developer	Base de datos	CLÍNICA KETEKURA

Contexto de negocio

La Clínica KETEKURA está automatizando procesos que hasta ahora se hacían a mano: el cálculo de descuentos, las asignaciones de los médicos y el control de pagos atrasados. Los bloques que ya escribiste en las semanas anteriores funcionan mientras los datos son los esperados. El problema es que en una base real los datos no siempre son los esperados.

En esta actividad vas a hacer que esos procesos sobrevivan a los datos raros. El primer ejercicio está resuelto y lo desarrollará el docente en clase, en tres partes.

Preparación del ambiente

1. Ejecuta el script crea_pobla_tablas_bd_CLINICA_KETEKURA.sql en tu esquema de trabajo.
2. Verifica la carga. Las consultas deben devolver 466 y 100:

```
SELECT COUNT(*) FROM ATENCION;  
SELECT COUNT(*) FROM PACIENTE;
```

3. Activa la salida por pantalla:

```
SET SERVEROUTPUT ON;
```

Sobre los acentos: el script trae caracteres acentuados que pueden verse alterados según la codificación de tu SQL Developer. No afecta a ningún ejercicio: los nombres de tablas y columnas no llevan acentos.

Entrega: guarda cada ejercicio en un archivo .sql independiente (EJ2E_apellido.sql, EJ3E_apellido.sql, EJ4E_apellido.sql) y súbelos al AVA al final de la sesión, con la salida obtenida.

Ejercicio 1 — Descuento de tercera edad

RESUELTO. Desarrollado por el/la docente en clase, en tres partes.

Requerimiento

La clínica aplica un descuento por tercera edad según tramos de edad definidos en la tabla PORC_DESCTO_3RA_EDAD. Construye un bloque PL/SQL anónimo que, para un paciente dado, muestre su nombre, su edad y el porcentaje de descuento que le corresponde.

Parte A: el bloque sin manejo

```
SET SERVEROUTPUT ON;

DECLARE
    v_run          PACIENTE.pac_run%TYPE := 7599430;
    v_nombre       VARCHAR2(60);
    v_edad         NUMBER(3);
    v_descuento    PORC_DESCTO_3RA_EDAD.porcentaje_desccto%TYPE;
BEGIN
    SELECT TRIM(pnombre || ' ' || apaterno),
           TRUNC(MONTHS_BETWEEN(SYSDATE, fecha_nacimiento) / 12)
    INTO   v_nombre, v_edad
    FROM   PACIENTE
    WHERE  pac_run = v_run;

    SELECT porcentaje_desccto INTO v_descuento
    FROM   PORC_DESCTO_3RA_EDAD
    WHERE  v_edad BETWEEN anno_ini AND anno_ter;

    DBMS_OUTPUT.PUT_LINE(v_nombre || ' (' || v_edad || ' años)');
    DBMS_OUTPUT.PUT_LINE('Descuento aplicable: ' || v_descuento || '%');
END;
/
```

Con el paciente **7599430** funciona: tiene 65 años y le corresponde un 2%.

Cambia el RUN por 6215470 y ejecuta de nuevo:

```
ORA-01403: no data found
ORA-06512: en línea 13
```

Ese paciente tiene 4 años. La consulta es correcta, pero no encuentra ningún tramo, y un SELECT ... INTO que no devuelve filas lanza NO_DATA_FOUND.

Parte B: manejar la excepción

```
DECLARE
    v_run          PACIENTE.pac_run%TYPE := 6215470;
    v_nombre       VARCHAR2(60);
    v_edad         NUMBER(3);
    v_descuento    PORC_DESCTO_3RA_EDAD.porcentaje_desccto%TYPE;
BEGIN
    SELECT TRIM(pnombre || ' ' || apaterno),
           TRUNC(MONTHS_BETWEEN(SYSDATE, fecha_nacimiento) / 12)
    INTO   v_nombre, v_edad
    FROM   PACIENTE
    WHERE  pac_run = v_run;

    SELECT porcentaje_desccto INTO v_descuento
    FROM   PORC_DESCTO_3RA_EDAD
    WHERE  v_edad BETWEEN anno_ini AND anno_ter;

    DBMS_OUTPUT.PUT_LINE(v_nombre || ' (' || v_edad || ' años)');
    DBMS_OUTPUT.PUT_LINE('Descuento aplicable: ' || v_descuento || '%');

EXCEPTION
```

```

    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('El paciente ' || v_run ||
                               ' no tiene descuento por tercera edad.');
```

END;
/

El paciente 6215470 no tiene descuento por tercera edad.

El bloque ya no se cae: informa y termina normalmente.

Experimento obligatorio: agrega una línea DBMS_OUTPUT.PUT_LINE('Sigo aqui'); justo después del segundo SELECT, dentro del cuerpo. Ejecuta con el paciente de 4 años. Ese mensaje no aparece. Explica en un comentario por qué. Si entiendes esto, entendiste el flujo de control de las excepciones.

Parte C: la red de seguridad

El bloque anterior maneja el caso que anticipamos. ¿Y los que no anticipamos?

```

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('El paciente ' || v_run ||
                               ' no tiene descuento por tercera edad.');
```

WHEN OTHERS THEN
 -- SIEMPRE al final
 DBMS_OUTPUT.PUT_LINE('Error inesperado al procesar el paciente.');

DBMS_OUTPUT.PUT_LINE('Codigo : ' || SQLCODE);
 DBMS_OUTPUT.PUT_LINE('Mensaje: ' || SQLERRM);

END;

Prueba a mover el WHEN OTHERS antes del WHEN NO_DATA_FOUND y compila. Anota el error que aparece y explica por qué la regla tiene sentido.

Cuatro cosas que debes retener

1. Una excepción ocurre en ejecución y muchas veces depende del dato, no del código.
2. Cuando salta, el control abandona el cuerpo y no vuelve: no es un IF.
3. La sección EXCEPTION va al final, antes del END, y no lleva BEGIN propio.
4. WHEN OTHERS va siempre al último, y debe informar con SQLCODE y SQLERRM.

Ejercicio 2 — Especialidad del médico

Dificultad: baja. Tiempo estimado: 9 minutos.

Requerimiento

El área de agenda necesita conocer la especialidad de un médico para asignarle atenciones. Construye un bloque PL/SQL anónimo que, para un RUN de médico dado, muestre su nombre completo y el nombre de su especialidad.

El bloque debe manejar de forma diferenciada estas tres situaciones, con un mensaje distinto para cada una:

- El médico existe y tiene exactamente una especialidad: se muestra la información.
- El médico tiene más de una especialidad: hay que avisar que no se puede determinar una sola y que se requiere revisión.
- El RUN no corresponde a ningún médico, o el médico no tiene especialidades registradas: hay que avisar que no se encontró información.

Además, agrega un WHEN OTHERS que informe el código y el mensaje del error.

Casos de prueba

RUN a probar	Qué debería ocurrir
3490261	Caso normal: el médico tiene una sola especialidad
5588583	Tiene dos especialidades registradas
99999999	No existe ningún médico con ese RUN

Pistas

- La especialidad sale de ESPECIALIDAD_MEDICO unida con ESPECIALIDAD por esp_id.
- Un SELECT ... INTO falla tanto con cero filas como con dos: son excepciones distintas y necesitan manejadores distintos.
- Verifica primero con una consulta SQL directa cuántas especialidades tiene cada uno de los tres RUN, para saber qué esperar.

Salida esperada (formato de referencia)

```
--- Medico 3490261 ---  
Margarita Cares - Especialidad: Inmunologia  
  
--- Medico 5588583 ---  
El medico tiene mas de una especialidad registrada.  
No es posible determinar una sola. Requiere revision.  
  
--- Medico 99999999 ---  
No se encontro informacion de especialidad para ese RUN.
```

Para pensar

Si el requerimiento cambiara a «mostrar todas las especialidades del médico», ¿seguirías usando excepciones para resolverlo? Escribe tu respuesta en un comentario.

Ejercicio 3 — Asignación por atenciones médicas

Dificultad: media. Este es el ejercicio central de la sesión. Tiempo estimado: 12 minutos.

Requerimiento

La clínica paga a sus médicos una asignación porcentual según la cantidad de atenciones que hayan realizado, de acuerdo con los tramos de la tabla TRAMO_ASIG_ATMED. Construye un bloque PL/SQL anónimo que recorra todos los médicos y, para cada uno, muestre:

- Su RUN y nombre completo.
- La cantidad de atenciones que realizó.
- El costo promedio de sus atenciones.
- El porcentaje de asignación que le corresponde según su tramo.
- El monto de la asignación, calculado como ese porcentaje sobre su sueldo base.

El problema

Los datos de la clínica tienen dos situaciones que van a romper el proceso:

1. Hay 21 médicos que no registran ninguna atención. Calcular su promedio implica dividir por cero.
2. Los tramos de la tabla llegan hasta 30 atenciones, y hay al menos un médico que supera ese número. Buscar su tramo no devuelve ninguna fila.

Restricción

El proceso debe recorrer los 51 médicos completos. Si falla con uno, debe informar el problema para ese médico y seguir con el siguiente. Un informe que se detiene en el médico número 3 no sirve.

Pistas

- Usa un cursor sobre MEDICO, como en las sesiones anteriores.
- Para que el proceso continúe, la sección EXCEPTION no puede estar en el bloque exterior: necesitas un BEGIN ... EXCEPTION ... END; anidado dentro del ciclo.
- Cuenta las atenciones y suma los costos en una sola consulta de grupo sobre ATENCION.
- Las dos excepciones que necesitas son ZERO_DIVIDE y NO_DATA_FOUND. Piensa cuál corresponde a cada situación antes de escribirlas.
- Agrega contadores para informar al final cuántos médicos se procesaron correctamente y cuántos quedaron con problemas.

Salida esperada (formato de referencia)

```
===== ASIGNACION POR ATENCIONES MEDICAS =====
3126425  Gregoria Gonzalez      19 aten.  prom 28.450   9%   72.728
5588583  Maria Fernandez             8 aten.  prom 31.200   7%   33.316
11959215 Texia Rojas                SIN ATENCIONES REGISTRADAS
7555800  Juana Perez                 36 aten.  prom 27.980  FUERA DE TRAMO
...
-----
Medicos procesados correctamente : 29
Medicos con incidencias          : 22
```

Nota: los nombres y montos dependen de tu carga de datos. Lo que se evalúa es que el proceso llegue hasta el último médico y que distinga las dos incidencias.

Ejercicio 4 — Registro de pagos morosos

Dificultad: alta. Ejercicio de estiramiento. Modifica datos. Tiempo estimado: 10 minutos.

Requerimiento

La clínica quiere poblar la tabla PAGO_MOROSO, que hoy está vacía, con las atenciones cuyo pago se hizo después de la fecha de vencimiento. Construye un bloque PL/SQL anónimo que:

1. Recorra los pagos de PAGO_ATENCION en que la fecha de pago es posterior a la fecha de vencimiento.
2. Para cada uno, obtenga los datos del paciente y el nombre de la especialidad de la atención.
3. Calcule los días de morosidad como la diferencia entre ambas fechas.
4. Calcule el monto de la multa como un 1% del monto de la atención por cada día de atraso.
5. Inserte la fila en PAGO_MOROSO.
6. Informe al final cuántas filas se insertaron y cuántas se omitieron por estar ya registradas.

El punto del ejercicio

Ejecuta el bloque dos veces seguidas. La segunda vez, PAGO_MOROSO ya contiene esas filas y su clave primaria es (pac_run, ate_id). Tu bloque debe capturar esa situación, no insertar el duplicado, contarlo aparte y seguir con la fila siguiente. No debe caerse ni detenerse.

Pistas

- La excepción que corresponde a una clave primaria repetida es DUP_VAL_ON_INDEX.
- Igual que en el ejercicio 3, el manejador va en un bloque anidado dentro del ciclo.
- Los días de atraso se obtienen restando fechas directamente: fecha_pago menos fecha_venc_pago devuelve un número de días.
- PAGO_MOROSO necesita el nombre completo del paciente, así que tendrás que unir PAGO_ATENCION con ATENCION, PACIENTE y ESPECIALIDAD.
- Cierra la transacción con COMMIT al final, fuera del ciclo.

Salida esperada (formato de referencia)

```
===== REGISTRO DE PAGOS MOROSOS =====  
Primera ejecucion:  
  Filas insertadas : 99  
  Ya registradas   : 0  
  
Segunda ejecucion:  
  Filas insertadas : 0  
  Ya registradas   : 99
```

Verificación y limpieza

```
SELECT COUNT(*) FROM PAGO_MOROSO;  
  
-- Para volver a empezar:  
DELETE FROM PAGO_MOROSO;  
COMMIT;
```

Desafío opcional

Vuelve al ejercicio 3 y agrégale trazabilidad. En vez de solo mostrar las incidencias por pantalla, acumúlalas para producir un resumen al final que indique, por separado:

1. Cuántos médicos no tienen atenciones registradas.
2. Cuántos médicos quedaron fuera de los tramos definidos.
3. Cuántos fallaron por una causa distinta, capturada por WHEN OTHERS, junto con el último SQLCODE registrado.

Después responde en un comentario: si este proceso corriera de madrugada sin nadie mirando la pantalla, ¿bastaría con DBMS_OUTPUT? ¿Dónde deberían quedar registradas las incidencias?

Autoevaluación

✓	Puedo?...
☐	Explicar la diferencia entre un error de compilación y una excepción.
☐	Escribir una sección EXCEPTION con varios manejadores y ubicarla correctamente.
☐	Explicar por qué el código posterior al punto de falla no se ejecuta.
☐	Reconocer, leyendo un bloque, cuál excepción predefinida va a producirse.
☐	Usar WHEN OTHERS con SQLCODE y SQLERRM, y decir por qué va al final.
☐	Usar un bloque anidado para que un ciclo continúe después de una fila con error.
☐	Distinguir cuándo un SELECT INTO lanza NO_DATA_FOUND y cuándo TOO_MANY_ROWS.

Para la próxima clase: Oracle tiene un nombre para el paciente que no cae en ningún tramo. Pero no tiene ninguno para el paciente que sí cae en un tramo y, aun así, no debería recibir el descuento porque la clínica decidió que solo aplica a ciertos sistemas de salud. ¿Cómo se avisa de un error que solo existe en las reglas del negocio? Esa es la sesión de excepciones definidas por el usuario.